Universidad Nacional Autónoma de México.
Facultad de Ciencias.

Computación distribuida.



Profesor: Fernando Michel Tavera.
Ayudante de laboratorio: Luis Mario Escobar Rosales .

Práctica 03

Recorridos BFS y DFS.
Fecha: 5 de Noviembre del 2026.

Alumnos:

Jesús Echeverría Verduzco	**No. Cuenta:** 321239167
José Carlos Pinzón Chan	**No. Cuenta:** 425122448

En esta práctica, el objetivo fue construir una topología de árbol para un sistema distribuido de topología inicial desconocida. La idea principal consistió en implementar nodos especializados (NodoDFS y NodoBFS) que, mediante el intercambio de mensajes por los canales, lograran asignar dinámicamente su nodo padre, sus nodos hijos y su distancia respecto a la raíz. Construir este tipo de topologías resulta extremadamente útil en los sistemas distribuidos, ya que permiten lograr una menor complejidad y evitar redundancias en la difusión de mensajes.

Detalles:

- **Como ejecutar:**

Para poder ejecutar este código se necesitan las bibliotecas *pytest* y *simply.* No obstante durante esta práctica, se tuvieron algunos problemas para que python pudiera encontrarlas una vez instaladas localmente. Es por ello que, para esta práctica, se utilizan entornos virtuales para la ejecución de los test.

A continuación se explica paso a paso cómo activar el entorno virtual, instalar las bibliotecas y ejecutar los test:

Suponiendo que se encuentra en la dirección donde guarda la carpeta Practica03 (no dentro de la carpeta de la práctica).

1. Abra una terminal en dicho lugar. Use el siguiente comando para crear el entorno virtual.

   ```
   python3 -m venv Practica03/src/venv
   ```

2. Para activar el entorno virtual utilice el comando:

   ```
   source Practica03/src/venv/bin/activate
   ```

3. Luego instale las bibliotecas *pytest* y *simply* en el entorno virtual:

   ```
   pip install pytest simpy
   ```

4. Ejecute el siguiente comando para iniciar los test:

```
pytest Practica03/src/Test.py
```

5. Una vez terminado, para desactivar el entorno virtual escriba:

```
deactivate
```

- **Implementación de BFS:**

La implementación del algoritmo de Anchura (BFS) en un sistema distribuido funciona mediante un modelo de paso de mensajes, donde la exploración avanza por niveles a partir de un nodo distinguido.
Dicho nodo referenciado como el nodo, propaga el mensaje, el cual lleva un conjunto de instrucciones que se realizan dentro del ambiente de Simpy. Una vez el mensaje llega desde el canal al buzón de entrada, el cual cada nodo posee de manera única, reaccionando una vez que llega el mensaje , y de dependiendo el tipo del mensaje se ejecuta una instrucción u otra.
Los mensajes se representan como diccionarios con las claves tipo, d, de y, en el caso de las respuestas, resp, de modo que el receptor siempre conoce el tipo de mensaje, la distancia que viaja y el nodo remitente.
Para la construcción del árbol, cada nodo gestiona un estado “tiene padre” de manera local que incluye parámetros en el mensaje como:

- Tipo de mensaje
- distancia de ese mensaje con respecto al nodo distinguido
- remitente

Una vez que el mensaje llega al nodo, los datos son procesados para determinar si el nodo ya fue explorado. Más aún, la variable local que permanece activa entre una iteración y otra del ciclo de escucha y se actualiza cuando el nodo adopta a su primer padre. A partir de ello la gráfica se construye de tal forma que la asignación de parentesco involucre su primera asignación o el camino más corto desde el nodo referenciado. En ambos casos el nodo reinicia su lista de hijos, calcula a qué vecinos debe preguntar (todos menos el remitente, mediante una lista por comprensión) y fija el número de respuestas que espera según el tamaño de esa lista. Si no tiene vecinos a quienes preguntar, responde de inmediato a su padre con una confirmación; en caso contrario, propaga un nuevo mensaje de exploración con su propia distancia a esos vecinos. Si el camino recibido no mejora el camino que ya tiene, el nodo contesta con un mensaje de rechazo dirigido únicamente al remitente.

Finalmente, sabemos que el algoritmo termina debido a que en algún momento los nodos actualizarán todos sus caminos con respecto al nodo referenciado hasta encontrar el más óptimo y después cualquier otra respuesta que provenga de otro padre será omitida; será entonces cuando vemos que el algoritmo incorpora un mecanismo de confirmación hacia atrás para determinar cuándo se terminó de construir el árbol. Cuando el contador llega a cero, el nodo ya conoce el resultado de toda su rama: si no es la raíz, envía una confirmación a su padre; si es la raíz, da por terminada la construcción del árbol generador de distancias mínimas.

- **Implementación de DFS:**

La implementación del algoritmo de Búsqueda en Profundidad (DFS) en este sistema distribuido opera mediante un modelo de paso de mensajes, donde el control de la exploración se transfiere secuencialmente de un nodo a otro.

Para mantener la jerarquía del árbol, cada nodo gestiona un estado local que incluye el identificador de su nodo padre (aquel que lo descubrió primero), una lista de los nodos que se han convertido en sus hijos, un registro de los canales hacia los vecinos que aún están pendientes por explorar y su nivel de profundidad, el cual se calcula dinámicamente al sumar uno a la distancia reportada por su padre.

El proceso de exploración comienza cuando el nodo raíz envía un mensaje de descubrimiento (`GO`) a su primer vecino disponible. Al recibir este mensaje por primera vez, un nodo asume inmediatamente al remitente como su padre, actualiza su distancia y propaga el mensaje hacia su propio *primer vecino*, cediendo el control para continuar la exploración en profundidad. Sin embargo, en una red distribuida es común que existan múltiples caminos hacia un mismo nodo; por ello, si un nodo que ya forma parte del árbol recibe un nuevo mensaje de exploración, contesta de inmediato con un mensaje de rechazo (`REJECT`). Esta regla es fundamental, ya que impide la formación de ciclos y asegura que la topología resultante sea estrictamente un árbol.

Finalmente, el algoritmo incorpora un mecanismo de retroceso o *backtracking* para garantizar que toda la red sea visitada. Cuando un nodo activo agota sus canales pendientes o sufre un rechazo, devuelve el control del algoritmo enviando un mensaje de retorno (`BACK`) a su nodo padre. Al recibir esta confirmación, el padre registra al emisor en su lista de hijos y retoma la exploración contactando a su siguiente vecino disponible. Este flujo de avance y retroceso se repite a lo largo de la red hasta que el control regresa al nodo raíz, el cual da por terminada la construcción del árbol recubridor una vez que ha procesado todos sus canales iniciales.